

Analysis and Detection of Android Stegomalware: the Impact of the Loading Stage

Diego Soi*
University of Cagliari
Cagliari, Italy
diego.soi@unica.it

Angelica Liguori
National Research Council of Italy
Rende, Italy
angelica.liguori@icar.cnr.it

Silvia Lucia Sanna*
University of Cagliari
Cagliari, Italy
silvia.l.sanna@unica.it

Leonardo Regano
University of Cagliari
Cagliari, Italy
leonardo.regano@unica.it

Giacomo Benedetti
National Research Council of Italy
Genova, Italy
giacomo.benedetti@ge.imati.cnr.it

Luca Caviglione
National Research Council of Italy
Genova, Italy
luca.caviglione@ge.imati.cnr.it

Giorgio Giacinto
University of Cagliari
Cagliari, Italy
Consorzio Interuniversitario
Nazionale per l'Informatica (CINI)
Roma, Italy
giorgio.giacinto@unica.it

Abstract

Due to the increasing use of advanced offensive techniques, the mitigation of Android malware is an urgent need. An emerging attack trend exploits steganography to conceal malicious payloads within applications to make attacks stealthier. Even if works on “stegomalware” are starting to emerge, they primarily focus on the multimedia part of the attack chain, i.e., on how to detect hidden data in images or videos. Therefore, this work aims at understanding whether the loading stage required for the extraction of cloaked information can generate detection signatures. To this aim, we develop a proof-of-concept implementation, which has been repacked within a real Android application and tested against several malware detection engines provided by VirusTotal. To anticipate possible offensive campaigns, we also performed tests by considering threat actors able to obfuscate the bytecode of the loader or the entire APK. Results indicate that standard tools are not ready to face stegomalware targeting Android applications. Therefore, we provide indications on how to improve forensics and attribution phases for Android malware endowed with information hiding capabilities.

CCS Concepts

• Security and privacy → Malware and its mitigation;

Keywords

Information Hiding, Stegomalware, Mobile Security, Android

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.
IH&MMSEC '25, San Jose, CA, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1887-8/25/06
<https://doi.org/10.1145/3733102.3733122>

ACM Reference Format:

Diego Soi, Silvia Lucia Sanna, Giacomo Benedetti, Angelica Liguori, Leonardo Regano, Luca Caviglione, and Giorgio Giacinto. 2025. Analysis and Detection of Android Stegomalware: the Impact of the Loading Stage. In *ACM Workshop on Information Hiding and Multimedia Security (IH&MMSEC '25)*, June 18–20, 2025, San Jose, CA, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3733102.3733122>

1 Introduction

To support a heterogeneous population of devices (e.g., smartphones and tablets) and to provide a vast array of features, Android implements several functionalities, which require a complex interplay of hardware and software components. Unfortunately, this leads to a multitude of vulnerabilities that can be exploited by a threat actor to inject malicious code, steal information, or cause overflows for executing arbitrary instructions [20]. To cope with the increasing amount of malware targeting Android, many mitigation techniques have been proposed, for instance a prime line of defense utilizes static analysis mechanisms [24]. However, to prevent detection and void the most efficient static analysis frameworks, an increasing attack trend is to cloak malicious payloads within various software assets, e.g., digital media [14]. An analysis of the most recent malware endowed with information hiding capabilities found that images are the preferred choice to embed malicious data, whereas only a minor fraction of threats exploits text or network traffic [51]. For the case of Android, a payload containing an attack routine or configuration data is cloaked through steganography within assets composing the application bundle. This type of malicious software has been named *stegomalware* [53].

Even if works addressing risks of stegomalware targeting Android are starting to emerge, they primarily focus on the multimedia part of the attack chain, i.e., on how to reveal the presence of hidden data in images or videos [7, 12, 17, 53]. Moreover, many works also consider a threat actor able to create a covert channel, i.e.,

a hidden communication path that can leak data between sandboxed processes [32]. Despite the adopted steganographic attack scheme, the literature largely neglects the steps needed to decode and use at runtime the cloaked information. In this perspective, evaluating the impact of the additional operations (e.g., in terms of bytecode or invoked syscalls) required to implement the *steganographic loading stage* is mandatory to fully understand the security implications of stegomalware acting in the wild. As an example, the stegomalware attack model could be unfeasible for some Android applications, e.g., due to anti-tampering mechanisms to prevent cloning or copyright infringements. At the same time, additional code may generate trivial signatures or may challenge the most popular detection schemes through obfuscation techniques.

Therefore, this work aims at investigating the creation of a steganographic loading stage and its repacking within real-world Android applications [16]. Specifically, we develop a basic loader for “decoding” the malicious payload hidden in multimedia assets of applications. Then, we quantify its impact by testing the performance of the most popular malware detection engines of VirusTotal (VT), i.e. an online service that inspects items with over 70 antivirus scanners. We provided also the obfuscated APKs. Since the use of steganography has several impacts also on digital forensics, understanding attribution implications is critical to update best practices [15].

Summing up, the contributions of this work are threefold: *i)* it assesses the feasibility of unpacking at run time payloads hidden within real Android applications, *ii)* it investigates the impact of various obfuscation approaches when used on stegomalware, and *iii)* it offers insights that could assist in the attribution process of Android stegomalware.

The rest of the paper is structured as follows. Section 2 provides a background on stegomalware and repacking of Android applications, while Section 3 outlines the threat model and the steganographic loading stage. Section 4 deals with the experimental setup, Section 5 showcases the obtained results, and Section 6 proposes ideas on how memory can help to spot the presence of a stegomalware. Section 7 reviews prior research on the mitigation of malware targeting Android and Section 8 concludes the paper.

2 Background

This section provides background information on stegomalware and Android applications relevant for the considered attack model. Specifically, Section 2.1 focuses on steganography, while Section 2.2 deals with the structure of applications and their execution model.

2.1 Steganography Techniques

As hinted, stegomalware employs steganographic techniques to evade signature-based detection, implement multi-stage payload delivery schemes, and exfiltrate sensitive data without being blocked or recognized by intrusion detection systems [14, 53]. Even if the literature abounds in mechanisms for cloaking data, the most commonly used technique is the Least Significant Bit (LSB), which directly embeds the information within the least significant bits of the color components of the pixels composing the carrier image. This technique has been adopted by different real-world malware samples, such as PNGLoader, RDAT, and RegDuke [51]. Beyond basic

LSB manipulation, some malware families implement advanced cloaking schemes that modify multiple bits of the color space, allowing a better trade-off between stealthiness and capacity [14]. To improve the resistance against forensics attempts, some threat actors further encode the hidden payload, for instance in Base64 [23].

In general, real-world attacks are endowed with simple steganographic techniques. On one hand, this allows not to inflate the loader, which is often implemented through a Microsoft Office macro or a scripting language [38]. On the other hand, a reduced complexity prevents lags that can reveal the presence of a running malicious software [32].

Therefore, we consider stegomalware using two LSB-based strategies to hide the various payloads within images. The first, denoted as *LSB plain*, embeds the information into the least significant bit of each color channel, i.e., red, green, and blue. The second strategy, denoted as *LSB variant*, has been observed in threats from the OceanLotus group and conceals the payload in the three least significant bits of the red and green channels and in the two least significant bits of the blue channel [14]. Along with the LSB plain and LSB variant techniques, we also implemented two distinct hiding patterns. The first is a *sequential* pattern that hides the payloads starting from the first pixels of the image, i.e., to model real-world malware. The second is a *squares* pattern that conceals the information within equally-sized blocks placed along the main diagonal of the image, i.e., to model an attacker trying to deploy a basic elusive behavior.

2.2 Android Application and Execution Models

With the aim of dropping a malicious payload within an Android application, an important aspect concerns its organization. Specifically, an Android application is written in Java/Kotlin and packed in the APK file format. The typical content is:

- a set of native code libraries written in C/C++ for interacting with the device, e.g., camera and microphone;
- dex files containing the executable code;
- `AndroidManifest.xml` with information about the permissions and components of the application;
- META-INF grouping signatures metadata;
- assets and resources such as multimedia files, e.g., images and audio. The former contains the compiled resources included in an `.arcs` file, while the latter regards non-compiled resources, which are stored in the `res` directory and referenced exclusively through an integer resource ID.

An Android APK can be decoded with tools like ApkTool¹ or Jadx² allowing to access the original source files in Smali. The extracted application contents can be modified by injecting or removing files (e.g., code, libraries, or resources) and rebuilt to prepare a *repacked* version of the original APK. The outcome is then *re-signed* with apksigner³ to allow its distribution and installation.

When launched, an Android application is forked by the Zygote process⁴ and initialized with pre-loaded classes, system libraries, and additional code. The application is finally loaded in

¹<https://apktool.org/>

²<https://github.com/skylot/jadx>

³<https://developer.android.com/tools/apksigner?hl=en>

⁴Zygote is the process responsible for forking new application processes to optimize app startup and memory management.

```

1 fun callDynamicApiByName(obj: Any,
2     functionName: String,
3     typeParam: Class<*>,
4     param: String) {
5     val method = obj::class.java.getMethod(functionName,
6         typeParam)
7     method.invoke(obj, param)
8 }
9
10 fun callStaticApiByName(className: String,
11     functionName: String,
12     vararg args: Any) {
13     val clazz = Class.forName(className)
14     val method = clazz.methods.find {
15         it.name == functionName }
16     if (method != null)
17         method.invoke(null, *args)
18 }
19 }

```

Listing 1: Kotlin code for dynamically executing static or non static APIs given their name, class and parameters.

memory starting from dex files, executed by the Android Runtime (ART), and possibly compiled with optimizations. On the contrary, dynamically-loaded Java/Kotlin objects are directly managed by the garbage collector and located in the Dalvik/ART heap. Additional data can be found in the memory without any corresponding file in the *anonymous mappings* section. The Android RAM also contains the stack section storing function call frames, local variables, and execution context for each thread.

An important group of local variables contained in the stack concerns strings representing Android APIs or Kotlin/Java functions belonging to a given class. This characterizes the use of *reflection*⁵, which allows the inspection of classes, interfaces, fields, and methods at runtime without knowing their names at compile time. For the case of implementing a stegomalware, the reflection mechanism can be used to execute the hidden payload, which is handled as a string. Listing 1 provides an example of reflection for the execution of the API contained in the `functionName` parameter.

3 Threat Model and Implementation

This section showcases the considered scenario of a stegomalware targeting the Android ecosystem. Specifically, Section 3.1 introduces the threat model, Section 3.2 describes the implementation of the steganographic loading stage, and Section 3.3 provides details on the obfuscation techniques.

3.1 Threat Model

The Android security model “*explicitly supports installation of apps from arbitrary sources, including alternative app stores and apps independently distributed without a store at all*” [37]. This enables an attacker to take advantage of the distribution of repacked malicious software components. Specifically, our threat model focuses on the last part of the development life cycle: the distribution phase [58]. In this phase, an application is released by the developer through an application store (e.g., Google Play Store, Aptoide, and Apkpure),

⁵Reflection is a feature of several programming language among which Java (<https://www.oracle.com/technical-resources/articles/java/javareflection.html>) and Kotlin (<https://kotlinlang.org/docs/reflection.html>).

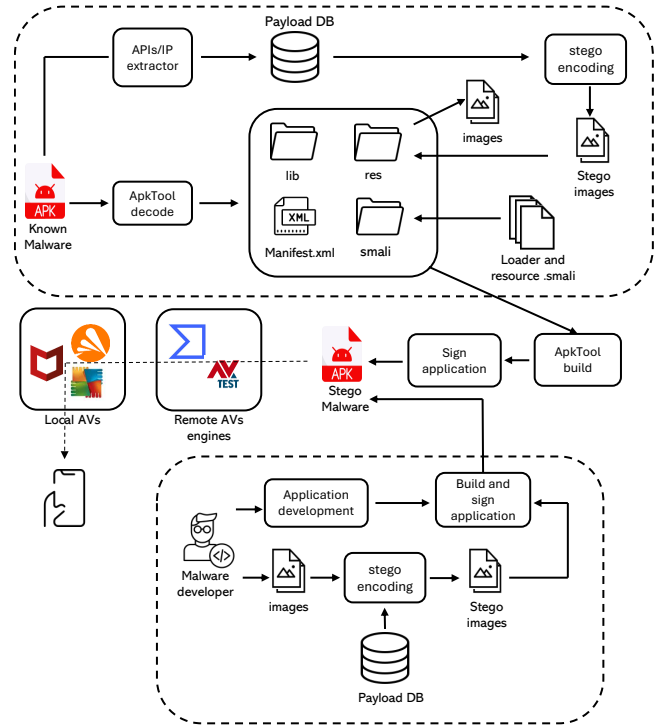


Figure 1: Threat model. Upper attack pipeline: a known malware is repacked to include steganographic payloads, as well as the needed loading and execution functionalities. Lower attack pipeline: a brand new malware is written from scratch and endowed with the aforementioned functionalities.

or by directly delivering the APK. This setting resembles a real scenario where the attacker may steal the credentials of the developer, “poison” a third-party library, or interfere with various stages of the software supply chain [36]. We also consider an attacker able to hide a malicious payload in the digital resources composing the application (i.e., images). For instance, the threat actor may want to evade signature-based detection mechanisms by cloaking via LSB steganography part of the offensive code or a specific pattern of API calls. To this aim, the attacker can exploit two entry points, as outlined in Figure 1. The first exploits a well-known application modified to contain the steganographic loading stage as well as the required malicious payloads. Here, the threat actor decompiles the application, hides the payload within a digital image, adds the compiled loader, and repacks the whole application, which is eventually delivered to end users. The second models the creation and distribution of an ad-hoc application containing the payload and the functionalities necessary for the deployment of the malware.

Despite the used entry point, the goal of the attacker is to extract the hidden payload at runtime to further advance in the offensive chain. In this work, we consider that the obtained payload is executed via reflection to bypass static detection mechanisms and increase the stealthiness of the attack [19, 34].

Algorithm 1 Pseudocode of the steganographic loading stage.**Require:** ID of the resource to employ**Ensure:** decoded is String

```

1: Initialize binary_message ← StringBuilder()
2: image ← LoadImage(ID)
3: height ← getHeight(image)
4: width ← getWidth(image)
5: for j=0 to height-1 do
6:   for i=0 to width-1 do
7:     pixel ← get(i, j)
8:     bits ← decoder(pixel)
9:     binary_message ← appendBits(bits)
10:  end for
11: end for
12: bytes_message ← chunked_8(binary_message)
13: decoded ← bytesToString(bytes_message)
14: return decoded

```

3.2 Steganographic Loading Stage

The steganographic loading stage extracts data from one or more application resources. With the aim of maintaining the code simple and preventing the introduction of additional signatures, we propose two different loaders.

The first is used to extract information hidden in RGB images and employs the `Bitmap` class from the `android.graphics` package providing a direct access to pixel values of RGB contents. For handling media with an alpha channel, i.e., a value used to control transparency, a different mechanism is needed. Thus, the second version of the loader handles RGBA and LA images through the `opencv` library. In essence, `opencv` provides APIs for manipulating image and video resources by treating them as matrices of pixels. In our implementation, the library is loaded inside the application context with its `initDebug()` function. In fact, since `opencv` is an imported third-party library, its execution requires to pass through the `initDebug()` API. Both loaders share the same extraction procedure.

Algorithm 1 showcases the pseudocode of the steps performed by the steganographic loading stage to retrieve the secret data from the digital media. As a preliminary step, the loader needs the ID of the image concealing the secret, which is its unique integer identifier set at build time. The loader also requires a string for storing the information extracted from the image. To this aim, the string `decoded` is created and initialized as a `StringBuilder` to contain a mutable sequence of characters. Then, the loader opens the image and obtains its boundaries. Such operations are reported in lines 2 – 4 and implemented with `getDrawable()` for handling a `Bitmap` object, or with the `imread()` function of `opencv` for RGBA and LA images. Despite the considered image format, lines 5 – 11 decode the cloaked data on a per-pixel basis, i.e., they create a binary representation of the LSB values containing the payload. The message is then organized in 8-bit long chunks and converted to their ASCII representation via the `bytesToString()` function. Finally, the extracted payload can be executed through reflection.

3.3 Obfuscation

Obfuscation aims to hinder malware detection systems by modifying the Smali code without affecting its semantics. The attacker can obfuscate the steganographic loading stage at compile time by acting over the build tool chain or operate on the already compiled application and obfuscate the APK.

For obfuscating the steganographic loading stage at compile time, we used features provided by the native R8 Android compiler, which offers mechanisms for renaming the methods while producing the Smali executable. Conversely, to obfuscate the final APK, we used the `obfuscapk` [3], which wraps 20 obfuscators.⁶ We selected the following `obfuscapk` permutations since the literature showcased that they are the most effective (see, e.g., [42] and the references therein):

- (1) arithmetic branch (Ab), reorder (R), method overload (Mo), call indirection (Ci), and goto insertion (Go);
- (2) reflection (Re), arithmetic branch, call insertion, and goto insertion;
- (3) reflection, call insertion, reorder, and method overload.

Figure 2 provides a fragment of the Smali code for the implementation of Algorithm 1 for RGB images and its obfuscated counterpart, when using permutation (1). As shown in Figure 2a, the main instructions can be easily identified, for instance, the `bitmap` parameter ruling the selection of the loading stage for RGB images and the `StringBuilder()` initializing the string that will contain the steganographic payload. Instead, Figure 2b reports the obfuscated version where the modifications reshaped the method. In detail, `obfuscapk` placed redirection instructions, such as `GOTO`, necessary to preserve the semantic of the application while instructions are syntactically shuffled. For example, the first instruction is `green()` despite it is only needed later to extract data from the green color channel. This example further highlights the disruption of the control flow graph of the application making the static analysis of an application challenging and error-prone. Dynamic analysis is required to investigate aspects of the execution that otherwise would pass unobserved.

4 Experimental Setup

This section details the setup used to perform our experiments. Specifically, Section 4.1 describes how applications have been prepared according to the considered threat models, whereas Section 4.2 showcases the experimental settings.

4.1 Creation of Test Applications

To model a threat actor exploiting a real application as depicted in the upper attack workflow of Figure 1, we repacked Telegram⁷ with our steganographic loading stage and the images containing the hidden payload. Such a choice has been driven by its popularity and the lack of APK protection mechanisms against software cloning [56]. In more detail, we prepared various images cloaking via LSB techniques the following payload:

```
textView.setText("This is a string set by an invocation
```

⁶The full list of obfuscators considered in this work is available on our companion repository.

⁷<https://telegram.org>

```

1 .method public final d1(Landroid/graphics/Bitmap;)Ljava/
  lang/String;
2 .registers 13
3 .param p1, "bitmap" # Landroid/graphics/Bitmap;
4
5 const-string v0, "bitmap"
6
7 invoke-static {p1, v0}, Lkotlin/jvm/internal/Intrinsics;
  ->checkNotNullParameter(Ljava/lang/Object;Ljava/lang/
  String;)V
8
9 .line 15
10 new-instance v0, Ljava/lang/StringBuilder;
11
12 invoke-direct {v0}, Ljava/lang/StringBuilder; -><init>()V
13

```

(a) Plain code of the d1 method used to load the Bitmap image.

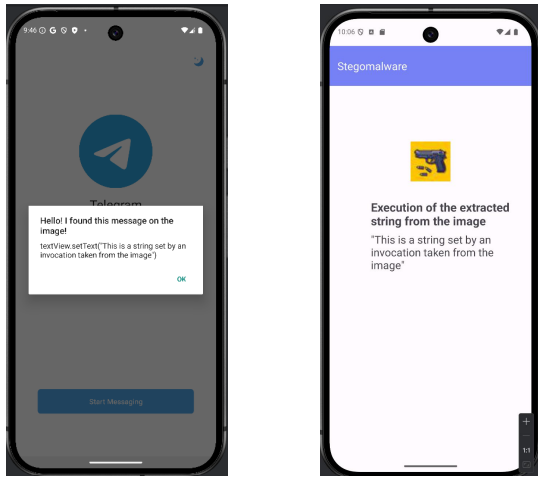
```

1 .method public final d1(Landroid/graphics/Bitmap;)Ljava/
  lang/String;
2 .registers 12
3
4 goto/32 :goto_f3
5
6 :goto_3
7 goto/32 :goto_7b
8
9 nop
10
11 :goto_7
12 invoke-static {v6}, Landroid/graphics/Color; ->green(I)I
13

```

(b) Obfuscated version of the d1 method.

Figure 2: An example showing the same Smali method reshaped through code obfuscation.



(a) Tampered real application (b) Custom tampered application

Figure 3: The hidden payload is extracted and executed on the two considered test applications.

taken from the image")

This is only a toy example with the purpose of investigating how a stegomalware works and how the hidden content can be processed by an Android application thanks to a loader extracting the steganographic content. For a better completeness, we tested the loading stage using the images of a previous work containing the payload of Necro malware⁸. Upon extraction, the string is then executed through reflection, and the result is depicted in Figure 3a. Although the embedded payload is benign, it was selected to focus the analysis on the ability of online detection engines to spot the main behaviors of stegomalware. This is not a limitation, since an API cannot be deemed inherently malicious without taking into account its execution context. For instance, this is the case of the

⁸<https://github.com/Ocram95/AndroidStego>

javax.crypto package APIs, which can be found both on ransomware and legitimate VPN applications. However, our approach is general and can be readily adapted to more realistic attack scenarios involving malicious payloads. As an example, previous work [50] has shown that it is feasible to embed the payload of Necro within Android APKs.

To prepare the test application, we did the following steps:

- (1) we prepared the loader in Java/Kotlin and we produced a mockup application to produce the needed Smali code. The obtained version of the loader has been then used throughout all the experiments;
- (2) we copied the image and the relevant files for the steganographic loading stage in the drawables, Smali, and lib folders of the unpacked Telegram application by using ApkTool;
- (3) we modified the original Telegram code to include the instructions to retain the image inside the resource space, e.g., defining its ID⁹, and the instructions to call the corresponding loader, i.e., Bitmap or openCV variant;
- (4) we re-built and re-signed the new Telegram application containing the stegomalware with ApkTool and apksigner, respectively.

The same steps were repeated to obtain the two obfuscated variants described in Section 3.3.

Instead, to model a threat actor preparing a custom malicious application as shown in the lower attack workflow of Figure 1, we developed an APK containing the steganographic loader as well as the required multimedia resources. The hidden payload is still the one used for the tampered Telegram application. Figure 3b depicts the final results obtained while the stegomalware is executed.

4.2 Testbed and Evaluation Methodology

To conduct experiments, we prepared different version of the applications presented in Section 4.1. Specifically, we obtained the following test sets:

⁹The file public.xml inside res directory contains the public resources' IDs that can be employed by the application.

- **Tampered Telegram application:** the application has been repacked with the two versions of the steganographic loading stage based on the `Bitmap` class and the `opencv` library. Contents have been hidden via the LSB plain sequential mechanism. Concerning the obfuscation, we produced three different variants, corresponding to the three permutations of `obfuscapk` discussed in Section 3.3. To have a fair comparison, we also produced a version of the application with the two steganographic loading stages obfuscated with R8;
- **Custom tampered application:** the application has been built by using again the two versions of the steganographic loading stage as well as obfuscated versions with the R8 and `obfuscapk` obfuscators, as presented in Section 3.3. Also, in this case, we only used images cloaking data with the LSB plain sequential mechanism.

Applications have been created and tested on an Intel i7-14650HX CPU @ 2.20 GHz with 32 GiB RAM under Microsoft Windows 11 Pro, using Android Studio 2024.2.2 and its built-in emulator running Android 15.0. At the time of our tests, we used the latest available version of Telegram, i.e., version 11.1.3¹⁰. For reproducibility purposes, we provide a companion repository¹¹ containing the code of applications, the loaders, and instructions to replicate our test methodology.

To check whether de-facto standard antiviruses are able to spot the presence of a stegomware, all the prepared applications have been submitted to VT. In our setting, this allowed to test 65 different detection engines, i.e. all the commercially-available antivirus engines in VT. To the best of our knowledge, none of them support the detection of malware endowed with steganographic features. Reports generated by VT have been collected and analyzed to check performances as well as to investigate potential Indicator of Compromise (IoC) linked with the different implementation of the steganographic loading stage. VT collects the results of performed analysis indexing them by unique hashes, which are provided in our companion repository. Furthermore, we recall that the goal of this work is understanding the impacts of the steganographic loading stage. In fact, differently from [12, 13, 23], we want to address the “binary” portion of a stegomware, rather than the possible signatures that the use of LSB steganography may left in the targeted digital media. For this reason, we solely investigated applications containing images cloaking data with the LSB plain and left the analysis of alternative hiding methods as a future research. Yet, the repository of this work contains all the assets modified with the various methods discussed in Section 2.1 to allow further experiments and make our results reproducible.

5 Experimental Results

This section describes the results of tests made with VT. Specifically, Section 5.1 presents the impact of the loader without obfuscation, Section 5.2 shows the results obtained by obfuscating the loader with R8, and Section 5.3 provides the results for the APKs obfuscated with `obfuscapk`.

	App	Obfuscation	Technique	Detections
Bitmap loader	Telegram	Vanilla	-	0/65 (*)
		R8	-	0/65 (*)
		obfuscapk	AbReMoCiGo	2/65
			RAbCiGo	3/65 (*)
	Custom app	Vanilla	-	0/65
		R8	-	2/65 (*)
		obfuscapk	AbReMoCiGo	2/65
			RAbCiGo	2/65
			RCiReMo	8/65

Table 1: Detection results of 65 VT engines for our custom application and the tampered Telegram when using the `Bitmap` library to load images. (*) denotes that VT used sandboxes.

5.1 Impact of the Vanilla Loader

The first round of experiments quantifies the impact of the steganographic loading stage when no obfuscation is used. In the following, we will refer to such a case as *vanilla*. To perform experiments, we submitted to VT the applications described in Section 4.2.

As shown in Tables 1 and 2, no detection engine is able to correctly identify the presence of the loader through static analysis. This means that suspicious API calls, signatures, or VT tags¹² used by the various engines are not effective for detecting a malware endowed with steganographic capabilities. In more detail, for the case of the custom tampered application, VT does not activate any of its Android sandboxes. This suggests that if dynamic analysis is not used, the vanilla steganographic loading stage is “transparent” to all the deployed detection engines. Instead, for the case of the tampered Telegram application, VT triggered the use of its Android sandboxes¹³ (i.e., VirusTotal R2DBox and Zenbox android) for performing the dynamic analysis. Then, VT correctly identified the tag related to reflection. However, this is a quite general indicator and could not be assumed as specific for the presence of a stegomware. Indeed, to check the reliability of results collected via VT, we also submitted the original, untampered Telegram application as a ground truth finding that it was marked with the reflection tag as well.

5.2 Impact of R8 Obfuscation

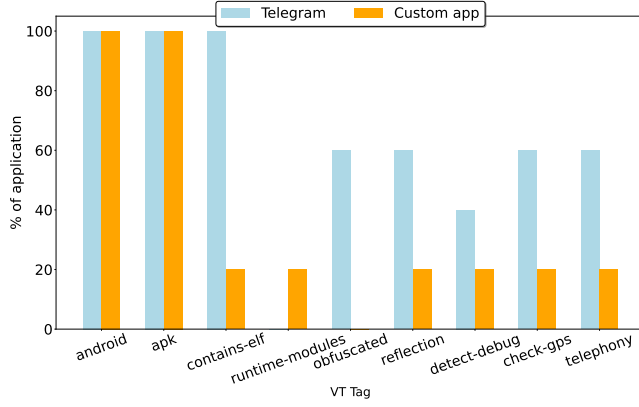
The second round of experiments assesses how the steganographic loading stage obfuscated with R8 impacts the performances of the detection. Referring again to Table 1 and 2, the obfuscation does not improve the detection rate. On one hand, obfuscation is intended to hinder the capability of analysis tools to spot suspicious code. On the other hand, it could serve as a trigger for more advanced testing mechanisms. Specifically, for the case of the custom tampered application, some of the tags outlined by VT do not correspond to its actual runtime behavior. This is the case of telephony and

¹⁰SHA256: 6c5e7eaa2a3039f1cd2d99e4a06692b43e6bf892b878a4e66ff86f7f34f65053

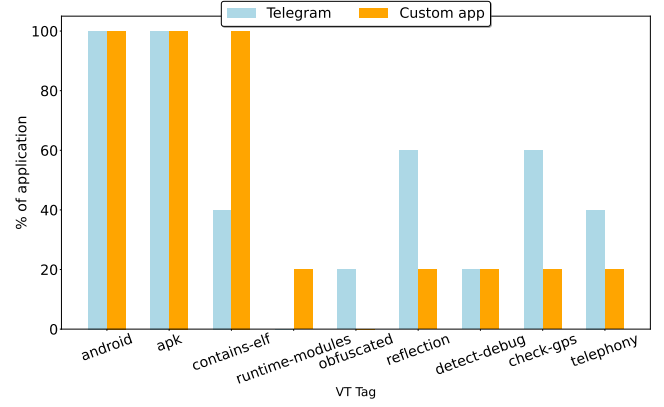
¹¹<https://github.com/Diesoi/AndroidStegoLoader/>

¹²The VT tags are listed in <https://docs.virustotal.com/docs/intelligence-tag-list>.

¹³The VT sandboxes are listed in <https://docs.virustotal.com/docs/in-house-sandboxes>.



(a) Percentage of APKs featuring the Bitmap loader variant for which VT marked with the specific tag.



(b) Percentage of APKs featuring the openCV loader variant for which VT marked with the specific tag.

Figure 4: Comparison of VT tags for different loader variants.

App		Obfuscation	Technique	Detections
openCV loader	Telegram	Vanilla	-	0/65 (*)
		R8	-	0/65 (*)
		obfuscapk	AbReMoCiGo	3/65
			RABCiGo	2/65 (*)
			RCiReMo	1/65
	Custom app	Vanilla	-	0/65
		R8	-	2/65 (*)
		obfuscapk	AbReMoCiGo	3/65
			RABCiGo	3/65
			RCiReMo	6/65

Table 2: Detection results of 65 VT engines for our custom application and the tampered Telegram when using the openCV library to load images. (*) denotes that VT used sandboxes.

check-gps that erroneously indicate that our application utilizes call or location services, which is not true. Only two engines flagged the custom application as malicious, i.e., Google and Ikarus. In both cases, our APK has been identified as a dropper, i.e., a malware downloading other components. As regards the tampered Telegram application, all the engines considered it as benign.

For both cases, VT also performed dynamic analysis via sandboxing. In particular, all the applications have been correctly flagged with the tags runtime-modules and reflection, denoting the presence of operations to load dynamic components and the use of code reflection, respectively.

5.3 Impact of Obfuscapk

The third round of experiments evaluates how VT performs when analyzing test applications obfuscated via obfuscapk. Also in this case, Table 1 and 2 portrait the result of the static analysis. In general, when obfuscating the whole APK, the detection rate slightly

increases. As regards the custom tampered application, VT reports two .dex files as malicious. The custom application has been also flagged as malicious by 8 engines when using the RCiReMo permutation. However, results were inconsistent since some engines labeled the sample as a trojan, a downloader or an adware, which is definitely not true.

For the case of the tampered Telegram application, there are not any relevant differences among the use of steganographic loading stages based on the Bitmap class and the openCV library for handling the images. Differently from the custom application, the tampered version of Telegram triggered the dynamic analysis mechanisms, i.e., the version using the RABCiGo obfuscation permutation with both Bitmap and openCV variants. Notably, only the Zenbox sandbox has been activated for these analyses, and its detection capabilities were limited to the identification of the reflection behavior.

5.4 Takeaways

As shown, VT largely fails to classify our applications as malicious. Independently by the library used to load the images within the steganographic loading stage, both custom and tampered Telegram applications are classified as benign by all the 65 malware detection engines. We recall that our test applications do not contain a “real” malicious payload, i.e., no harm is done neither to the running device nor the user. However, we expected to observe an higher detection rate from VT engines, since a potential attack chain is employed, as described in the threat model in Section 3. This points out that *VT cannot assess the presence of a stegomalware most of the time*. Despite VT embodies sandboxes to conduct dynamic analysis of applications, these are not consistently used across all the submitted samples¹⁴. Yet, we note that *dynamic analysis does not improve the detection trend, raising concerns on the techniques involved by VT engines*.

¹⁴VT does not provide documentation on the use of dynamic analysis sandboxes, then we cannot arbitrarily trigger them.

The mechanism used for loading the images does not affect VT results. Despite the openCV is a well-known and vastly used Android library, the lack of awareness is alarming. Since the use of third-party library does not represent a characterizing factor for malware investigation, a more aggressive and feature-rich library can be used to facilitate the extraction of the payload from a digital media asset and implement more complex steganographic techniques.

Concerning obfuscation, only few engines used by VT flag the applications as malware. Specifically, the Ikarus engine shows a consistent trend in classifying obfuscated application as Trojan-Dropper.AndroidOS or Trojan-Downloader.AndroidOS [27, 28]. These families of malware are responsible to collect and drop other malware or advertising systems in the device. Since this classification is limited to a very restricted number of malware detection engines, we assume that either obfuscation provides engines with information to conduct a more serious analysis or it acts as a deceptive factor triggering an alarm without providing actual IoCs.

Lastly, an important remark concerns the observed tags. Specifically, Figure 4a and 4b depict the tags provided by VT over the entire set of tested applications grouped according to the mechanisms for loading the image, i.e., Bitmap or openCV. As shown, the library used in the loading stage does not influence the tags produced by the analysis. Moreover, the tags are very generic and hardly provide clues for further investigation of the applications. Detecting the presence of a stegomalware through such an analysis should be then considered as ineffective.

5.5 Towards Stegomalware Identification

Since the various VT engines failed to *classify* our test applications as malicious, there is the need of rethinking part of the *attribution* strategy to face such a class of threats. In general, difficulties could be ascribed to malware detection engines, which have been designed to cover a wide-range of offensive techniques primarily targeting code, libraries, configuration files, and seldom digital media. However, another layer of complexity is given by the deceptive nature of stegomalware, delivering attacks as a combination of multiple steps and stages [8, 14]. In our scenario, the steganographic loading stage implements a class to read a resource from the application filesystem, collects the information from the pixels of the media, and uses reflection to execute the acquired data. To detect the malware, this would require to *i*) raise alerts due to the loading of images into memory, *ii*) trigger stegomalware-related static analysis algorithms, and lastly *iii*) implement specific dynamic analysis methodologies able to classify the payload as malicious. This multi-stage attack chain then requires developers to tune their engines without exceeding in triggering false positives, since the first steps of the chain may be legitimate in most cases. The used malware detection engines demonstrated to be largely ineffective in the identification of IoCs for stegomalware. For instance, Telegram has been flagged with reflection, despite this programming technique is legitimate [26]. Therefore, IoCs dealing with reflection should not be considered valid for stegomalware identification.

Concerning obfuscation, our findings suggest that it is not inherently a trigger for malware classification and can serve as an

effective evasion technique against static analysis. However, obfuscation should be considered as a double-edged sword, since massive modifications may raise alarms in malware detection systems. Therefore, the complete obfuscation of an APK (e.g., via obfuscapk) can represent an IoC even if not specific of a stegomalware.

Finally, existing literature on stegomalware classification reports a precise taxonomy for the information hiding in applications [15]. By using this taxonomy, it would be possible to produce more accurate and comprehensive IoCs that may better address the attribution of stegomalware. As an example, inspection of media assets [12, 33] should become a routinary task, possibly by also considering other IoCs, such as discrepancies in file formats and EXIF data [8]. Besides, implementing specific dynamic analysis methodologies is necessary to confirm the malicious behavior of the application containing a steganographic loader. As it will be detailed in Section 6, memory analysis techniques appear promising.

6 Detection Via Memory Forensics

This section discusses the proposed attribution methodology based on dynamic and memory analysis approaches. Specifically, Section 6.1 introduces the implemented method and Section 6.2 details the main findings.

6.1 Memory Forensics Methodology

As discussed in Section 5.5, dynamic analysis should be further explored to support the identification of stegomalware especially if jointly applied with static approaches. This can be used to implement an attribution mechanism to extract the hidden payload from the memory regions of the application.

To this aim, we leverage Frida¹⁵ and Fridump¹⁶. The former is an instrumentation framework for dynamic analysis allowing to intercept API calls. The latter is a Python script for reading the raw bytes of the memory space (e.g., stack, heap, and dex files). The tool Frida¹⁷ can run either on a real or an emulated device with root permissions to have a full control over the target application¹⁸. In our setting, we installed and executed the frida-server on the Android device, which has been attached to the target application as a privileged service. This allowed the creation of a bridge between the device and the host to hook APIs or dump the memory via Fridump.

An alternative technique to access the RAM relies upon LiMe [25], which provides full memory dumps without distinguishing among applications. A limit of LiMe is the need of recompiling the whole kernel to have a file map of the memory, defined as a profile. Since we are only interested to inspect the RAM of a specific application, such an approach has not been used in this work. Still, the main challenge is rooted in the volatile nature of the RAM: the time at which the dump happens is then crucial.

¹⁵<https://frida.re/docs/home/>

¹⁶<https://github.com/Nightbringer21/fridump>

¹⁷<https://frida.re/docs/android/>

¹⁸In case of non-rooted device, the frida-gadget library can be included in the target APK through repacking techniques to serve as a bridge between the host and the application.


```

1  ...
2  0x3CF058: res/drawable-hdpi/fb_ic_checkmark_original_
   LA_seq_lsb.png
3  ...
4  0x3ED9C8: [116, 101, 120, 116, 86, 105, 101, 119, 46,
   115, ...]
5  ...
6  0x3EDB54: textView.setText("This is a string set by
   an invocation taken from the image")
7  ...
8

```

Listing 2: Memory dump of the tampered Telegram application when data is hidden in a LA image, with the path of the resource, the sequence of bytes of the hidden payloads, and its cleartext.

6.2 Memory Analysis

To understand if a stegomalware can be identified at runtime through the presence in memory of the payload hidden in the various digital pictures, we slightly modified the APKs of test applications to avoid possible biases. Specifically, we removed the popup features depicted in Figure 3 displaying the payload, i.e., the applications have been pruned of any Log and Message Box functionality. Our GitHub page shows all the steps for the Memory Forensics acquisition and analysis.

Listing 2 reports a fragment of the collected memory dump for the tampered version of Telegram. As shown, the dump contains three major details. The first is the path of the resource, which can be used to identify the digital media that the attacker used to conceal the payload. The second is the “raw” representation of the payload in bytes and can be passed through another process for running detection algorithms [12]. The third is the cleartext string residing within the Dalvik Heap Main Space that contains dynamically-allocated objects.

The dump can be considered an effective starting point for the design of automatic assessment tools as well as forensics/attribution processes. Upon loaded in RAM, multimedia files can be further processed. For instance, paths and APIs can be checked against a knowledge-base of signatures, patterns, or known assets. Slices of the dump can be parsed through a regex to search for known invocations, IP ranges, and known malicious DNS-entries. Alas, directly inspecting the RAM has some limitations. The first concerns the need of acting “close in time” with the activation of the steganographic loading stage. In fact, time-bomb mechanisms delaying the execution of a given stage of the attack chain may turn out the approach unpractical [32, 39]. Similarly, aggressive use of CPU registers or `malloc()/free()` operations could make the secret payload to remain “dumpable” for very short time periods. Another major limitation is that inspecting the RAM is only suitable to spot cleartext payloads. Lastly, if the stegomalware deploys some evasion mechanisms to prevent the execution in a sandbox [15], the presented approach is still viable but requires sorting to more sophisticated tools, e.g., LiMe.

7 Related Works

Android malware analysis is gaining momentum owing to the diffusion of malicious software and the increasing adoption of Android,

even in critical infrastructures. In general, the literature proposed three main detection approaches. The first is the *static analysis* based on the evaluation of specific strings (e.g., IPs and URLs), code patterns, or permissions defined within the manifest file [5, 43]. The second is the *dynamic analysis* aimed at extracting certain API invocations and network flows during the execution of a program [11, 29]. The third is the *hybrid analysis* that combines the aforementioned techniques [6, 55]. More recently, other approaches have started to emerge. Specifically, *signature-based* methods designed to identify unique patterns [48] and *AI-based* approaches relying on Machine Learning [46] and Deep Learning [21].

However, several analysis and detection solutions can be challenged by emerging malicious software such as fileless malware [35, 54] as well as by anti-analysis tools such as sandbox detection, anti-debugging, anti-reverse engineering protection [2, 45]. In this vein, detection approaches based on *memory forensics* have been recently proposed as a workaround. In essence, memory forensics exploits features directly extracted from dumps of the RAM, see, e.g., [4, 10]. Such techniques can be also applied to Android malware detection [25, 41]. In more detail, new approaches are based on the “pure” RAM analysis [30] or exploit specific encodings such as grayscale/RGB images transformation [1]. Memory forensics turned out to be very efficient for identifying specific malware categories as well as the presence of cleartext data and encryption keys [22, 40]. Another approach increasingly used to prevent malware detection leverages code obfuscation [52, 57], where several techniques are applied to make the decompilation task harder.

An emerging aspect for the mitigation of stegomalware concerns the steps needed to enforce some form of attribution. Specifically, the digital objects used to implement part of the steganographic attack chain need to be collected and analyzed to spot hidden data or define the type of threat [8]. The attribution process can guide forensics experts and also support the design of countermeasures or mitigation techniques, especially against stegomalware targeting digital pictures and audio [18]. However, the majority of work focuses on the media part of the information hiding process and neglects all the artifacts caused by the presence of code, runtime behaviors, or temporary data, that can help distill signatures for spotting stegomalware [8, 31].

Concerning approaches for the mitigation of malware endowed with some form of information hiding capabilities, the work in [9] presents a proxy-based architecture for detecting HTML in-line objects containing data hidden via LSB steganography. A variant of LSB is at the basis of the Invoke-PSImage threat, which exhibits major statistical signatures in the tampered images leading to straightforward detection mechanisms [49]. However, many threats do not use steganography but rely upon various digital manipulation techniques, e.g., the content is stored at the beginning/end of a media as it happens in MiniDuke. In this case, detection can reduce to search for inconsistencies in the size of the file or in the bytes of the preamble/trail sequence of the image [44]. When detection is not possible, a valuable workaround is sanitization, i.e., the disruption of the hidden data without impairing the carrier. Sanitization is important to avoid that a malware operates correctly, e.g., by disrupting information on the IP addresses to contact [5, 43]. An increasing trend is to exploit AI-based approaches, such as neural architectures, for removing malicious payloads [33, 47].

Lastly, about the totality of works dealing with Android security tends to not consider the use of steganography for building complex attack chains. A major exception is the seminal work in [53] investigating whether steganography can be used to distribute malicious contents via the Google Play store. The work in [17] is a recent attempt to assess how steganography can endanger Android security. However, it considers the embedding of a complete malware, which is a use-case never observed in the wild, as stegomalware tends only to cloak small bits of data to reduce its footprint [14, 38, 51].

8 Conclusion and Future Works

In this paper, we advanced in the understanding of stegomalware targeting Android. Specifically, we investigated the impact of the steganographic loading stage, which is often neglected in favor of the multimedia part of the attack chain. Overall, our investigation showcased that malware detection engines used by VT do not explicitly flag the presence of a loading stage with any IoC. This lack of awareness is alarming, since IoCs are crucial for the attribution of a malicious software. We also found that obfuscation slightly impacts the detection, thus opening up to a plethora of possible future attack scenarios, e.g., to evade static controls through more “aggressive” approaches. Therefore, a more precise definition of IoCs for behaviors characterizing a stegomalware is necessary for its correct detection and attribution.

Part of our ongoing research aims at refining the memory analysis approach presented in this work. Specifically, it could be tested against a large-scale population of applications, mainly to better understand how the Android ecosystem is permeable to stegomalware. In particular, this could be better addressed with a full RAM acquisition, that is, studying the effect of the stegomalware on the whole system, rather than analyzing the single target process. Moreover, we plan to investigate the impact on the detection of different loading designs, e.g., by deploying more complex steganographic functions for the decoding. Another part of our research activity is devoted to investigate the impact of real payloads on the various malware detection engines as well as online application stores.

Acknowledgments

This work was partially supported by Project SERICS (PE00000014) under the NRRP MUR program funded by the EU - NGEU.

This work was carried out while Silvia Lucia Sanna was enrolled in the Italian National Doctorate on Artificial Intelligence run by Sapienza University of Rome in collaboration with the University of Cagliari.

References

- [1] Aisha Ali-Gombe, Sneha Sudhakaran, Ramyapandian Vijayakanthan, and Golden G. Richard. 2023. cRGB_Mem: At the Intersection of Memory Forensics and Machine Learning. *Forensic Science International: Digital Investigation* 45 (2023). doi:10.1016/j.fsidi.2023.301564
- [2] Mohammed K. Alzaylaee, Suleiman Y. Yerima, and Sakir Sezer. 2017. Emulator vs Real Phone: Android Malware Detection Using Machine Learning. *Proceedings of the 3rd ACM International Workshop on Security and Privacy Analytics, co-located with CODASPY 2017* (2017), 65 – 72. doi:10.1145/3041008.3041010
- [3] Simone Aonzo, Gabriel Claudiu Georgiu, Luca Verderame, and Alessio Merlo. 2020. Obfuscapk: An Open-Source Black-Box Obfuscation Tool for Android Apps. *SoftwareX* 11 (2020), 100403. doi:10.1016/j.softx.2020.100403
- [4] James M. Aquilina, Eoghan Casey, and Cameron H. Malin. 2008. *Malware Forensics: Investigating and Analyzing Malicious Code*. 1 – 674 pages. doi:10.1016/B978-1-59749-268-3.X0001-1
- [5] Daniel Arp, Michael Spreitzerbarth, Malte Hübner, Hugo Gascon, and Konrad Rieck. 2014. Drebin: Effective and Explainable Detection of Android Malware in Your Pocket. *21st Annual Network and Distributed System Security Symposium* (2014). doi:10.14722/ndss/2014.23247
- [6] Saba Arshad, Munam A. Shah, Abdul Wahid, Amjad Mehmood, Houbing Song, and Hongnian Yu. 2018. SAMADroid: A Novel 3-Level Hybrid Malware Detection Model for Android Operating System. *IEEE Access* 6 (2018), 4321 – 4339. doi:10.1109/ACCESS.2018.2792941
- [7] Shikha Badhani and Sunil K Muttou. 2018. Evading Android Anti-malware by Hiding Malicious Application Inside Images. *International Journal of System Assurance Engineering and Management* 9 (2018), 482–493. doi:10.1007/s13198-017-0692-7
- [8] Bernhard Birnbaum, Christian Kraetzer, and Jana Dittmann. 2023. Stego-Malware Attribution: Simple Signature and Content-based Features Derived and Validated from Classical Image Steganalysis on Five Exemplary Chosen Algorithms. In *17th International Conference on Emerging Security Information, Systems and Technologies*.
- [9] Jorge Blasco, Julio Hernandez-Castro, José María de Fuentes, and Benjamin Ramos. 2012. A Framework for Avoiding Steganography Usage Over HTTP. *Journal of Network and Computer Applications* 35, 1 (2012), 491–501. doi:10.1016/j.jnca.2011.10.003
- [10] Ahmet Selman Bozkir, Ersan Tahillioglu, Murat Aydos, and Ilker Kara. 2021. Catch them Alive: A Malware Detection Approach through Memory Forensics, manifold Learning and Computer Vision. *Computers and Security* 103 (2021). doi:10.1016/j.cose.2020.102166
- [11] Iker Burguera, Urko Zurutuza, and Simin Nadjim-Tehrani. 2011. Crowddroid: Behavior-based Malware Detection System for Android. *Proceedings of the ACM Conference on Computer and Communications Security* (2011), 15 – 25. doi:10.1145/2046614.2046619
- [12] Nunziato Cassavia, Luca Caviglione, Massimo Guarascio, Giuseppe Manco, and Marco Zuppelli. 2022. Detection of Steganographic Threats Targeting Digital Images in Heterogeneous Ecosystems Through Machine Learning. *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications* 13, 3 (2022), 50–67. doi:10.22667/JOWUA.2022.09.30.050
- [13] Cassavia, Nunziato and Caviglione, Luca and Guarascio, Massimo and Liguori, Angelica and Surace, Giuseppe and Zuppelli, Marco. 2023. Federated Learning for the Efficient Detection of Steganographic Threats Hidden in Image Icons. In *Pervasive Knowledge and Collective Intelligence on Web and Social Media*. Springer Nature Switzerland, 83–95. doi:10.1007/978-3-031-31469-8_6
- [14] Luca Caviglione and Wojciech Mazurczyk. 2022. Never Mind the Malware, Here's the Stegomalware. *IEEE Security & Privacy* 20, 5 (2022), 101–106. doi:10.1109/MSEC.2022.3178205
- [15] Luca Caviglione, Steffen Wendzel, and Wojciech Mazurczyk. 2017. The Future of Digital Forensics: Challenges and The Road Ahead. *IEEE Security & Privacy* 15, 6 (2017), 12–17. doi:10.1109/MSP.2017.4251117
- [16] Xiao Chen, Chaoran Li, Derui Wang, Sheng Wen, Jun Zhang, Surya Nepal, Yang Xiang, and Kui Ren. 2020. Android HIV: A Study of Repackaging Malware for Evading Machine-Learning Detection. *IEEE Transactions on Information Forensics and Security* 15 (2020), 987 – 1001. doi:10.1109/TIFS.2019.2932228
- [17] Danilo Dell'Orco, Giorgio Bernardinetti, Giuseppe Bianchi, Alessio Merlo, and Alessandro Pellegrini. 2024. Would You Mind Hiding My Malware? Building Malicious Android Apps with Stegopack. *SSRN* (2024). doi:10.2139/ssrn.5039499
- [18] Jana Dittmann, Christian Kraetzer, Jost Alemann, and Bernhard Birnbaum. 2024. Forensic Trace Analysis for MP3 based Stego-Malware: Exemplary Study for Stego-Algorithm and Capacity Attribution to derive YARA Rules for Malware Identification. In *Proceedings of the 2024 ACM Workshop on Information Hiding and Multimedia Security*. 101–112. doi:10.1145/3658664.3659641
- [19] Parvez Faruki, Rati Bhan, Vinesh Jain, Sajal Bhatia, Nour El Madhoun, and Rajendra Pamula. 2023. A Survey and Evaluation of Android-based Malware Evasion Techniques and Detection Frameworks. *Information* 14, 7 (2023). doi:10.3390/info14070374
- [20] Shivi Garg and Niyati Baliyan. 2021. Comparative Analysis of Android and iOS from Security Viewpoint. *Computer Science Review* 40 (2021), 100372. doi:10.1016/j.cosrev.2021.100372
- [21] M. Gopinath and Sibi Chakkaravarthy Sethuraman. 2023. A Comprehensive Survey on Deep Learning based Malware Detection Techniques. *Computer Science Review* 47 (2023). doi:10.1016/j.cosrev.2022.100529
- [22] Tobias Groß, Marcel Busch, and Tilo Müller. 2021. One Key to Rule Them All: Recovering the Master Key from RAM to Break Android's File-based Encryption. *Forensic Science International: Digital Investigation* 36 (2021). doi:10.1016/j.fsidi.2021.301113
- [23] Massimo Guarascio, Marco Zuppelli, Nunziato Cassavia, Luca Caviglione, and Giuseppe Manco. 2022. Revealing MageCart-like Threats in Favicons via Artificial Intelligence. In *Proceedings of the 17th International conference on Availability, Reliability and Security*. 1–7. doi:10.1145/3538969.3544437
- [24] Hashida Haidros Rahima Manzil and S. Manohar Naik. 2024. Detection Approaches for Android Malware: Taxonomy and Review Analysis. *Expert Systems with Applications* 238 (2024). doi:10.1016/j.eswa.2023.122255

- [25] Andri Heriyanto, Craig Valli, and Peter Hannay. 2015. Comparison of Live Response, Linux Memory Extractor (LiME) and Mem Tool for Acquiring Android's Volatile Memory in the Malware Incident. *Australian Digital Forensics Conference* (2015), 5 – 14. doi:10.4225/75/57b3f143fb884
- [26] Ulf Kargén, Noah Mauthe, and Nahid Shahmehri. 2023. Characterizing the Use of Code Obfuscation in Malicious and Benign Android Apps. In *Proceedings of the 18th International Conference on Availability, Reliability and Security (ARES '23)*. Association for Computing Machinery, Article 27, 12 pages. doi:10.1145/3600160.3600194
- [27] Kaspersky Lab. 2025. Trojan-Downloader.AndroidOS.Agent. <https://threats.kaspersky.com/en/threat/Trojan-Downloader.AndroidOS.Agent/>.
- [28] Kaspersky Lab. 2025. Trojan-Dropper.AndroidOS.Agent. <https://threats.kaspersky.com/en/threat/Trojan-Dropper.AndroidOS.Agent/>.
- [29] David Sean Keyes, Beiqi Li, Gurdip Kaur, Arash Habibi Lashkari, Francois Gagnon, and Frederic Massicotte. 2021. EntropLyzr: Android Malware Classification and Characterization Using Entropy Analyzer of Dynamic Characteristics. *2021 Reconciling Data Analytics, Automation, Privacy, and Security: A Big Data Challenge* (2021). doi:10.1109/RDAAPS48126.2021.9452002
- [30] Saneeha Khalid and Faisal Bashir Hussain. 2024. VolMemDroid—Investigating Android Malware Insights with Volatile Memory Artifacts. *Expert Systems with Applications* 253 (2024). doi:10.1016/j.eswa.2024.124347
- [31] Stefan Kiltz, Jana Dittmann, Fabian Loewe, Christian Heidecke, Max John, Jonas Mädel, and Fabian Preißler. 2024. Forensic Image Trace Map for Image-Stego-Malware Analysis: Validation of the Effectiveness with Structured Image Sets. In *Proceedings of the 2024 ACM Workshop on Information Hiding and Multimedia Security*. 125–130. doi:10.1145/3658664.3659659
- [32] Jean-Francois Lalande and Steffen Wendzel. 2013. Hiding Privacy Leaks in Android Applications Using Low-attention Raising Covert Channels. In *2013 International Conference on Availability, Reliability and Security*. IEEE, 701–710. doi:10.1109/ARES.2013.92
- [33] Angelica Liguori, Marco Zuppelli, Daniela Gallo, Massimo Guarascio, and Luca Caviglione. 2024. Erasing the Shadow: Sanitization of Images with Malicious Payloads Using Deep Autoencoders. In *International Symposium on Methodologies for Intelligent Systems*. Springer, 115–125. doi:10.1007/978-3-031-62700-2_11
- [34] Davide Maiorca, Davide Ariu, Igino Corona, Marco Aresu, and Giorgio Giacinto. 2015. Stealth Attacks: An Extended Insight into the Obfuscation Effects on Android Malware. *Computers & Security* 51 (2015), 16–31. doi:10.1016/j.cose.2015.02.007
- [35] Steve Mansfield-Devine. 2017. Fileless Attacks: Compromising Targets without Malware. *Network Security* 2017, 4 (2017), 7 – 11. doi:10.1016/S1535-4858(17)30037-5
- [36] Jefferson Martínez and Javier M Durán. 2021. Software Supply Chain Attacks, a Threat to Global Cybersecurity: SolarWinds' Case Study. *International Journal of Safety and Security Engineering* 11, 5 (2021), 537–545. doi:10.18280/ijss.110505
- [37] René Mayrhofer, Jeffrey Vander Stoep, Chad Brubaker, and Nick Kralevich. 2021. The Android Platform Security Model. *ACM Transactions on Privacy and Security* 24, 3, Article 19 (April 2021), 35 pages. doi:10.1145/3448609
- [38] Wojciech Mazurczyk and Luca Caviglione. 2015. Information Hiding as a Challenge for Malware Detection. *IEEE Security & Privacy* 13, 2 (2015), 89–93. doi:10.1109/MSP.2015.33
- [39] Aaron Mos and Md Minhaz Chowdhury. 2020. Mobile Security: A Look into Android. In *2020 IEEE International Conference on Electro Information Technology*. IEEE, 638–642. doi:10.1109/EIT48999.2020.9208339
- [40] Tilo Müller and Michael Spreitzenbarth. 2013. FROST: Forensic Recovery of Scrambled Telephones. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 7954 (2013), 373 – 388. doi:10.1007/978-3-642-38980-1_23
- [41] Hamad Naeem, Shi Dong, Olorunjube James Falana, and Farhan Ullah. 2023. Development of a Deep Stacked Ensemble with Process based Volatile Memory Forensics for Platform Independent Malware Detection and Classification. *Expert Systems with Applications* 223 (2023). doi:10.1016/j.eswa.2023.119952
- [42] Francesco Pagano, Luca Verderame, and Alessio Merlo. 2024. Obfuscating Code Vulnerabilities Against Static Analysis in Android Apps. In *ICT Systems Security and Privacy Protection*. Springer Nature Switzerland, 381–395. doi:10.1016/j.jisa.2022.103311
- [43] Ya Pan, Xiuting Ge, Chunrong Fang, and Yong Fan. 2020. A Systematic Literature Review of Android Malware Detection Using Static Analysis. *IEEE Access* 8 (2020), 116363 – 116379. doi:10.1109/ACCESS.2020.3002842
- [44] Damian Puchalski, Luca Caviglione, Rafał Kozik, Adrian Marzecki, Sławomir Krawczyk, and Michał Choraś. 2020. Stegomalware Detection Through Structural Analysis of Media Files. In *Proceedings of the 15th International Conference on Availability, Reliability and Security*. 1–6. doi:10.1145/3407023.3409187
- [45] Siegfried Rasthofer, Steven Arzt, Marc Miltenberger, and Eric Bodden. 2016. Harvesting Runtime Values in Android Applications That Feature Anti-Analysis Techniques. *23rd Annual Network and Distributed System Security Symposium* (2016). doi:10.14722/ndss.2016.23066
- [46] Zahoor-Ur Rehman, Sidra Nasim Khan, Khan Muhammad, Jong Weon Lee, Zhihan Lv, Sung Wook Baik, Peer Azmat Shah, Khalid Awan, and Irfan Mehmood. 2018. Machine Learning-Assisted Signature and Heuristic-based Detection of Malwares in Android Devices. *Computers and Electrical Engineering* 69 (2018), 828 – 841. doi:10.1016/j.compeleceng.2017.11.028
- [47] Preston K. Robinette, Hanchen D. Wang, Nishan Shehadeh, Daniel Moyer, and Taylor T. Johnson. 2023. SUDS: Sanitizing Universal and Dependent Steganography. In *European Conference on Artificial Intelligence*, Vol. 372. doi:10.3233/FAIA230489
- [48] Andrea Saracino, Daniele Sgandurra, Gianluca Dini, and Fabio Martinelli. 2018. MADAM: Effective and Efficient Behavior-based Android Malware Detection and Prevention. *IEEE Transactions on Dependable and Secure Computing* 15, 1 (2018), 83 – 97. doi:10.1109/TDSC.2016.2536605
- [49] Andreas Schaffhauser, Wojciech Mazurczyk, Luca Caviglione, Marco Zuppelli, and Julio Hernandez-Castro. 2022. Efficient Detection and Recovery of Malicious PowerShell Scripts Embedded into Digital Images. *Security and Communication Networks* 2022, 1 (2022), 4477317. doi:10.1155/2022/4477317
- [50] Diego Soi, Silvia Lucia Sanna, Angelica Liguori, Marco Zuppelli, Leonardo Regano, Davide Maiorca, Luca Caviglione, Giuseppe Manco, and Giorgio Giacinto. 2025. On the Feasibility of Android Stegomalware: A Detection Study. *Italian Conference on CyberSecurity* (2025). To appear.
- [51] Fabian Strachanski, Denis Petrov, Tobias Schmidbauer, and Steffen Wendzel. 2024. A Comprehensive Pattern-based Overview of Stegomalware. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*. 1–10. doi:10.1145/3664476.3670886
- [52] Guillermo Suarez-Tangil, Santanu Kumar Dash, Mansour Ahmadi, Johannes Kinder, Giorgio Giacinto, and Lorenzo Cavallaro. 2017. DroidSieve: Fast and Accurate Classification of Obfuscated Android Malware. *CODASPY 2017 - Proceedings of the 7th ACM Conference on Data and Application Security and Privacy* (2017), 309 – 320. doi:10.1145/3029806.3029825
- [53] Guillermo Suarez-Tangil, Juan Tapiador, and Pedro Peris-Lopez. 2014. Stegomalware: Playing Hide and Seek With Malicious Components in Smartphone Apps. In *International Conference on Information Security and Cryptology*. 496–515. doi:10.1007/978-3-319-16745-9_27
- [54] Sudhakar and Sushil Kumar. 2020. An Emerging Threat Fileless Malware: a Survey and Research Challenges. *Cybersecurity* 3, 1 (2020). doi:10.1186/s42400-019-0043-x
- [55] Fatma Taher, Omar AlFandi, Mousa Al-kfairy, Hussam Al Hamadi, and Saed Alrabaa. 2023. DroidDetectMW: A Hybrid Intelligent Model for Android Malware Detection. *Applied Sciences* 13, 13 (2023). doi:10.3390/app13137720
- [56] Simon Tanner, Ilian Vogels, and Roger Wattenhofer. 2020. Protecting Android Apps from Repackaging Using Native Code. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 12056 (2020), 189 – 204. doi:10.1007/978-3-030-45371-8_12
- [57] Danish Vasan, Mamoun Alazab, Sobia Wassan, Hamad Naeem, Babak Safaei, and Qin Zheng. 2020. IMCFN: Image-based Malware Classification Using Fine-tuned Convolutional Neural Network Architecture. *Computer Networks* 171 (2020). doi:10.1016/j.comnet.2020.107138
- [58] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, et al. 2024. Research Directions in Software Supply Chain Security. *ACM Transactions on Software Engineering and Methodology* (2024). doi:10.1145/3714464